0/6 Questions Answered

Project initial implementation - training team member

Q1 Joint responsibilities

1 Point

Please upload:

A pair of JSON files representing

- one input sample (with real representative values)
- and one model output (again, with real representative values).

(Training, serving, and data team members must agree on this item together; training and inference team members ingest an input like the sample and produce an output like the sample, data team member produces something like the input sample in online and offline workflows.)

If your project involves more than one model, you will have one JSON pair per model. In that case, please name each JSON clearly to indicate which model it "belongs" to.

Input JSON sample(s):

Please select file(s) Select file(s)

Output JSON samples(s):

Please select file(s) Select file(s)

(4-person team only) If you are part of a four-person team, also upload

- a table enumerating all the containers involved in each role, with links to their Dockerfiles/Docker Compose files, and a link to the equivalent K8S manifest for each one.

The object is to show that each of the individual role-owned systems will be supported by the DevOps/Platform role, although at this stage team members are working independently.

> **Upload in PDF format.**

Please select file(s) Select file(s)

Save Answer

Q2 Training contributions

9 Points

Delivering a trained model file and some training code is not enough: this course is about *operationalizing* ML processes, including training. That means another engineer (or the TA) can reproduce a training run, inspect tracking information for a run, and compare candidates (including simpler baselines).

Requirements to get credit for those deliverables:

- All training runs should be executed on Chameleon from inside a container on a compute instance, and should be tracked in MLFlow. (For purposes of this project, "local" work doesn't count.)
- No one-off training scripts for different configurations: structure your code so that candidates and hyperparameters are selected via configuration (a single configuration dictionary that you edit directly in your code, a standalone config file in JSON or YAML format that is read in by your training code, or configuration specified by command line arguments). You should have one training script (or if your model options include totally separate frameworks, like one scikit-learn and one Pytorch model, one training script per framework). (If your feature involves multiple models with different prediction tasks, then you'll have one training script per framework pre prediction task.)
- For each run, you must log configuration parameters, model quality metrics that are appropriate for the prediction task, model training cost metrics (e.g. time per epoch, total training time), and information about the training environment (e.g. GPU information).
- You should give your "manager" (me! I'm your manager!) choices for managing the tradeoff between training/serving speed, cost, and model quality. The table should include at least 1 simple baseline model, as well as other candidates you want to consider.
- Use reasonable and well-justified strategies for hyperparameter tuning. (Note: even if you are not using Ray, there are other good libraries for hyperparameter tuning beyond grid search.)

Q2.1 Training runs table

4 Points

Create a table of training runs with good candidates clearly marked; each row links to an MLflow run. An example of a few rows of such a table is below.

Candidate	MLflow run link	Code version	Key hyperparams	Key model metrics	Key training cost metrics	Notes
baseline	http://...	`sha`	`...`	`...`	`...`	establishes baseline
v1	http://...	`sha`	`...`	`...`	`...`	tradeoff: better X, worse Y
v2	http://...	`sha`	`...`	`...`	`...`	why it is promising + next experiment

You will highlight the rows that you consider most promising, and in the notes, explain why (e.g. one model has the best accuracy, a different model has accuracy almost as good but is much faster to train, etc.).

> **Upload in PDF format with clickable links. You must test each link and make sure that it works.**

Please select file(s) Select file(s)

Note: There is no credit for systems that are not running on Chameleon Cloud. In this table, you should *only* include training runs that you executed on Chameleon compute instances.

Save Answer

Q2.2 Repository artifacts
3 Points

Please upload training artifacts:

Dockerfile(s) for the training container (and optionally, another Dockerfile for interactive development):

Please select file(s) Select file(s)

Training code as a Python script (not an interactive notebook), and sample training config file(s) if config lives in a separate file.

Please select file(s) Select file(s)

If you have multiple trained models, you may upload several training scripts; please name files carefully to distinguish which is which.

Note: There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. You should *only* submit items that you have implemented and executed on Chameleon.

Save Answer

Q2.3 Demo video
1 Point

Upload a sped-up demo video: one complete training run in a Docker container on Chameleon. (If training takes hours, you can record in snippets including beginning, middle, and end.)

The video should be no longer than a few minutes.

Please select file(s) Select file(s)

Note: There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. You should *only* submit items that you have implemented and executed on Chameleon.

Save Answer

Q2.4 Live MLFlow service
1 Point

Provide a link to your live MLflow service running on Chameleon, browseable by course staff, with all your training runs.

Set up the lease so that this MLFlow service is "up" until midnight on Tuesday, April 7. If your team has signed up for a project presentation, you should also bring your MLFlow service up on the day of the presentation.

Save Answer

Q3 Bonus

0 Points

If you are attempting the extra credit:

Use Ray Train's integration with your training framework to real effect (I don't mean calling `ray submit` on an unmodified training script) in a way that goes beyond what we had done in the lab. To get bonus credit, you must show through a concrete example how your integration makes training more robust.

Submit a PDF document explaining in detail, including the concrete realistic example, and also any artifacts (e.g. Dockerfiles, Python scripts) that go along with it.

Please select file(s) Select file(s)

Note: There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. You should *only* submit items that you have implemented and executed on Chameleon.

Save Answer

Save All Answers

Submit & View Submission